

Experiment no.8

Aim : To understand the Static Analysis SAST process and learn to integrate Jenkins SAST to SonarQube.

Theory :

1. Introduction to Static Application Security Testing (SAST):

Static Application Security Testing (SAST) is a **white-box testing technique** used to identify vulnerabilities in the source code, bytecode, or binary files of an application **without executing** the program.

It helps developers detect security flaws early in the **Software Development Life Cycle (SDLC)** — typically during the coding phase — thus saving time, effort, and cost compared to fixing vulnerabilities after deployment.

2. Working Principle of SAST:

- The SAST tool **scans the source code** or compiled code of an application.
 - It **analyzes data flows**, control flows, and dependencies to find issues such as:
 - SQL Injection
 - Cross-Site Scripting (XSS)
 - Buffer Overflows
 - Hard-coded passwords or secrets
 - Unvalidated inputs and insecure APIs
 - The tool then provides **reports with line numbers, severity levels, and remediation suggestions**.
-

3. Advantages of SAST:

- Detects security flaws **early** in development.
- Does not require running the application (can test incomplete code).
- Integrates easily with CI/CD pipelines for **continuous security checks**.
- Improves **code quality** and adherence to **secure coding standards**.

4. SonarQube for Static Analysis:

SonarQube is an open-source platform used for **continuous inspection of code quality**. It performs **static code analysis** to detect:

- Bugs
- Code smells (poor design choices)
- Vulnerabilities and security hotspots

SonarQube supports multiple programming languages such as **Java, C++, Python, JavaScript, PHP**, and more. It provides a **dashboard** that visualizes code quality metrics, coverage, and risk areas, helping teams maintain clean and secure codebases.

5. Jenkins Integration with SonarQube (SAST Integration):

Jenkins, a popular **Continuous Integration (CI)** tool, can be used to **automate the SAST process** by integrating it with **SonarQube**.

Steps involved:

1. **Install and Configure SonarQube:**
 - Set up the SonarQube server.
 - Generate an authentication token for Jenkins.
 2. **Install SonarQube Plugin in Jenkins:**
 - Go to *Manage Jenkins* → *Plugins* → *Available Plugins* → *SonarQube Scanner*.
 3. **Configure SonarQube in Jenkins:**
 - Add SonarQube server details and token under *Manage Jenkins* → *Configure System*.
 4. **Set Up Jenkins Pipeline/Job:**
 - Add a build step: “Execute SonarQube Scanner.”
 - Define the sonar-project.properties file in your project repository.
 5. **Run the Build:**
 - Jenkins will trigger SonarQube analysis during each build.
 - The analysis report is uploaded to the SonarQube dashboard automatically.
-

6. Benefits of Jenkins–SonarQube Integration:

- Enables **automated static analysis** in CI/CD pipelines.

- Provides **real-time feedback** to developers about code security and quality.
- Helps enforce **secure coding practices** and compliance.
- Reduces manual effort and ensures **consistent quality checks** across builds.

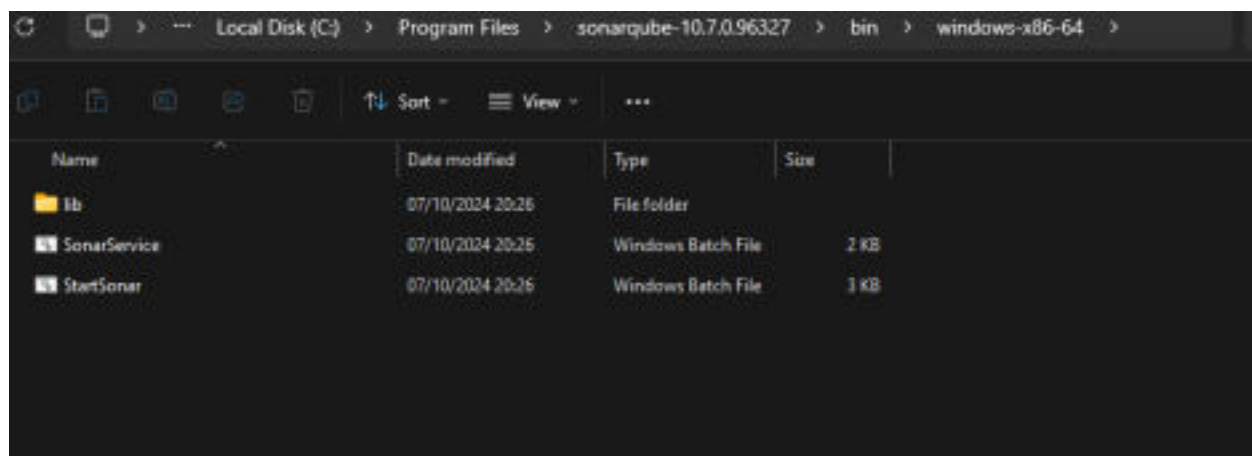
Procedure:

Step 1 :- Go To <https://www.sonarsource.com/products/sonarqube/downloads> and download Community Editions.

The screenshot shows the SonarQube Downloads page with four editions:

- Community Edition:** Free and open source for productivity & code quality. Download for free. Features include static code analysis for 20+ languages and frameworks, detection of issues in AI-generated code, and SonarQube server runs in Docker.
- Developer Edition:** Essential capabilities for small teams & businesses. Download. Features include additional languages (C, C++, Obj-C, Swift, ABAP, T-SQL, and PL/SQL), AutoConfig for C and C++ projects, Taint analysis with deeper SAST for Java, C#, JavaScript, and TypeScript, and detection of advanced bugs causing runtime errors and crashes in Python & Java.
- Enterprise Edition:** Designed to meet Enterprise requirements. Download. Features include additional languages (Apex, COBOL, JCL, PL/I, RPG, and VB6), unlimited integrations with DevOps platforms, security engine custom configuration for more powerful taint analysis, custom rules to detect private secret patterns, and aggregate projects and applications into a single scan.
- Data Center Edition:** For high availability, scalability, & performance. Download. Features include autoscaling in a Kubernetes cluster, component redundancy, data resiliency, horizontal scalability, high performance under extreme load, standard commercial support included, and 24/7 white glove service.

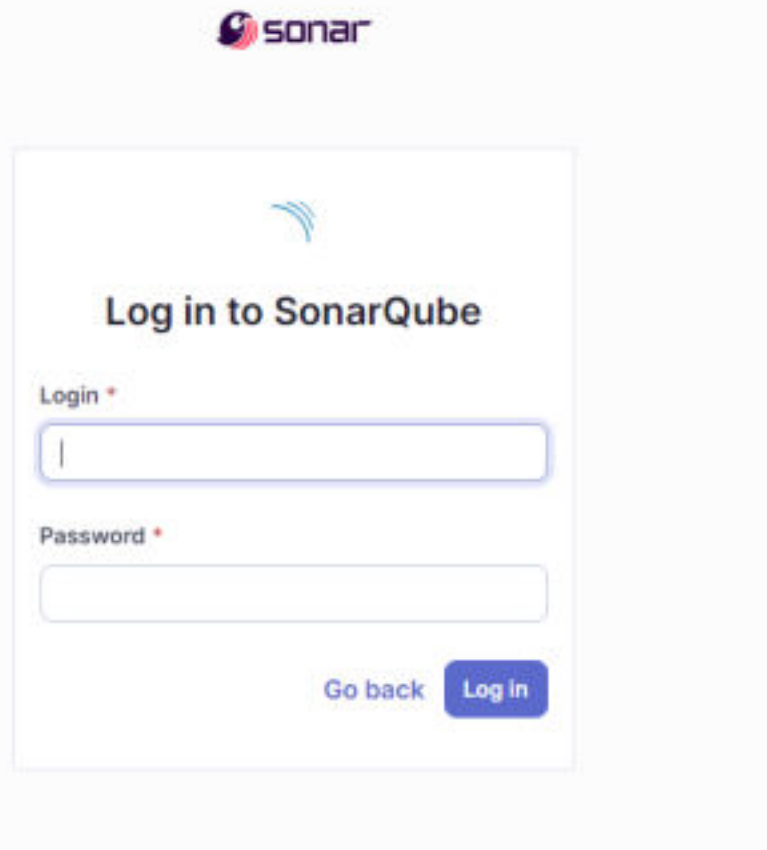
Step 2 :- Now Go To bin/windows-x86-64 and start StartSonar



Step 3 :- You Will See Process Is Up & Sonarqube Is Operational

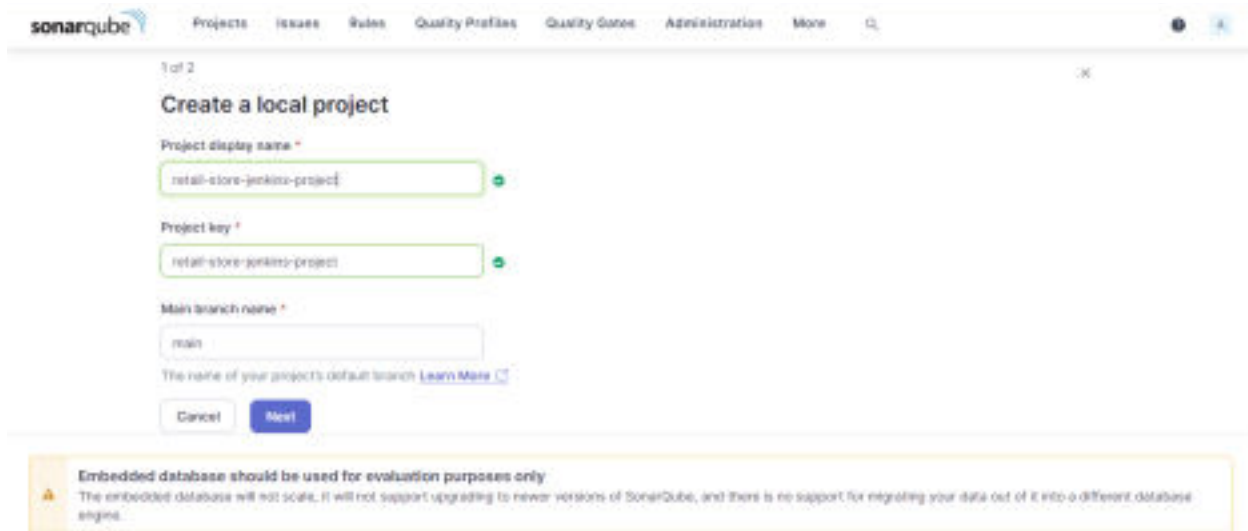
```
73685properties
WARNING: A terminally deprecated method in java.lang.System has been called
WARNING: System::setSecurityManager has been called by org.sonar.process.PluginSecurityManager (file:/C:/Program Files
/sonarqube-10.7.0.96327/lib/sonar-application-10.7.0.96327.jar)
WARNING: Please consider reporting this to the maintainers of org.sonar.process.PluginSecurityManager
WARNING: System::setSecurityManager will be removed in a future release
2024.10.07 21:15:04 INFO app[[o.s.a.SchedulerImpl] Process[web] is up
2024.10.07 21:15:04 INFO app[[o.s.a.ProcessLauncherImpl] Launch process[COMPUTE_ENGINE] from [C:\Program Files\sonarqu
be-10.7.0.96327]: C:\Program Files\OpenLogic\jdk-17.0.12.7-hotspot\bin\java -Djava.awt.headless=true -Dfile.encoding=UTF
-8 -Djava.io.tmpdir=C:\Program Files\sonarqube-10.7.0.96327\temp -XX:-OmitStackTraceInFastThrow --add-opens=java.base/ja
va.util=ALL-UNNAMED --add-exports=java.base/jdk.internal.ref=ALL-UNNAMED --add-opens=java.base/java.lang=ALL-UNNAMED --a
dd-opens=java.base/java.nio=ALL-UNNAMED --add-opens=java.base/sun.nio.ch=ALL-UNNAMED --add-opens=java.management/sun.man
agement=ALL-UNNAMED --add-opens=jdk.management/com.sun.management.internal=ALL-UNNAMED -Xmx512m -Xms128m -XX:+HeapDumpOn
OutOfMemoryError -Dhttp.nonProxyHosts=localhost[127.*][:1] -cp ./lib/sonar-application-10.7.0.96327.jar;C:\Program File
s\sonarqube-10.7.0.96327\lib\jdbc\h2\h2-2.2.224.jar org.sonar.ce.app.CeServer C:\Program Files\sonarqube-10.7.0.96327\te
mp\sq-process16549977733649395237properties
2024.10.07 21:15:05 WARN app[[startup] #####
#####
2024.10.07 21:15:05 WARN app[[startup] Default Administrator credentials are still being used. Make sure to change the
password or deactivate the account.
2024.10.07 21:15:05 WARN app[[startup] #####
#####
WARNING: A terminally deprecated method in java.lang.System has been called
WARNING: System::setSecurityManager has been called by org.sonar.process.PluginSecurityManager (file:/C:/Program Files
/sonarqube-10.7.0.96327/lib/sonar-application-10.7.0.96327.jar)
WARNING: Please consider reporting this to the maintainers of org.sonar.process.PluginSecurityManager
WARNING: System::setSecurityManager will be removed in a future release
2024.10.07 21:15:09 INFO app[[o.s.a.SchedulerImpl] Process[ce] is up
2024.10.07 21:15:09 INFO app[[o.s.a.SchedulerImpl] SonarQube is operational
```

Step 4:- Go to <http://localhost:9000> and login to Sonarqube



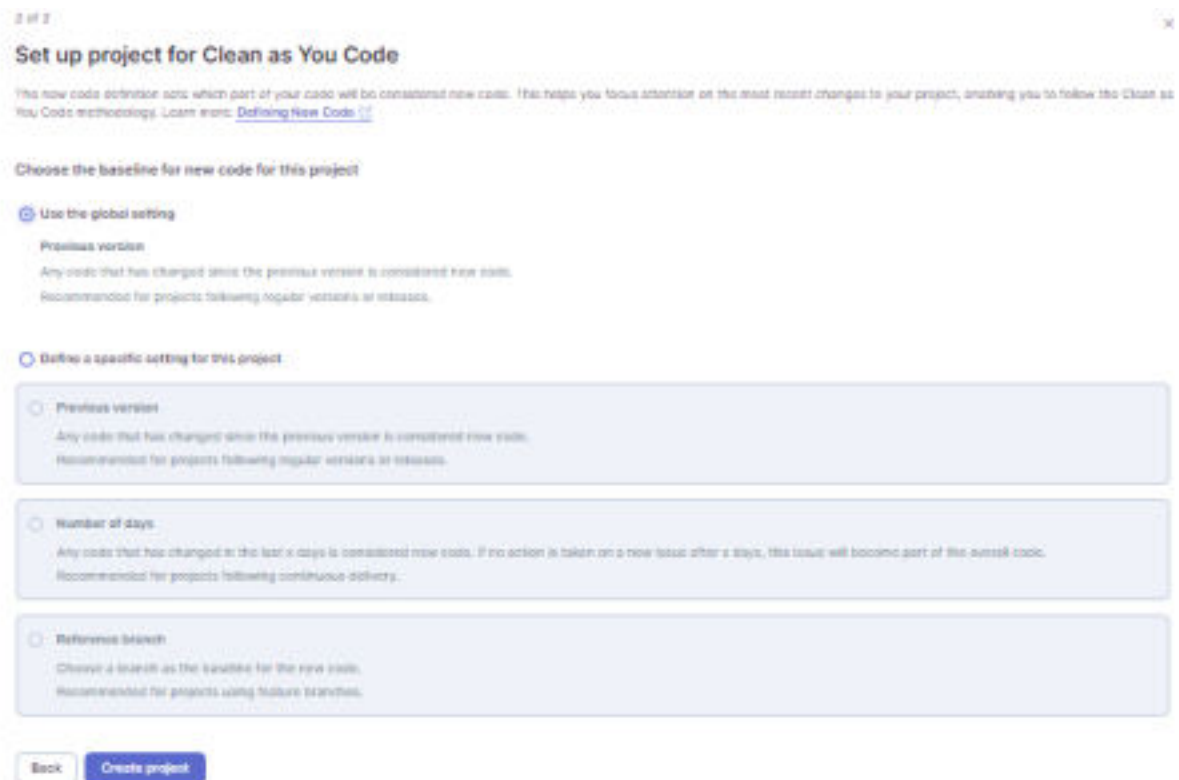
The image shows the SonarQube login page. At the top, there is the Sonar logo. Below it, the text "Log in to SonarQube" is displayed. Underneath, there are two input fields: "Login *" and "Password *". The "Login *" field contains a single character, possibly a pipe symbol. At the bottom right, there are two buttons: "Go back" and "Log In".

Step 5 :- Create A Local Project.



The screenshot shows the 'Create a local project' form in SonarQube. The form has three input fields: 'Project display name' with the value 'retail-store-jenkins-project', 'Project key' with the value 'retail-store-jenkins-project', and 'Main branch name' with the value 'main'. Below these fields is a link 'Learn More' and two buttons: 'Cancel' and 'Next'. At the bottom of the form, there is a yellow warning box with an icon of a triangle and an exclamation mark. The text in the warning box reads: 'Embedded database should be used for evaluation purposes only. The embedded database will not scale, it will not support upgrading to newer versions of SonarQube, and there is no support for migrating your data out of it into a different database engine.'

Step 6 :- Choose Global Setting As Baseline



The screenshot shows the 'Set up project for Clean as You Code' form in SonarQube. The form has a title 'Set up project for Clean as You Code' and a subtitle 'This new code definition sets which part of your code will be considered new code. This helps you focus attention on the most recent changes to your project, enabling you to follow the Clean as You Code methodology. Learn more: [Defining New Code](#)'. Below the subtitle is a section 'Choose the baseline for new code for this project' with two radio buttons. The first radio button is selected and labeled 'Use the global setting'. Below this radio button is a section 'Previous version' with the text 'Any code that has changed since the previous version is considered new code. Recommended for projects following regular versions or releases.' The second radio button is labeled 'Define a specific setting for this project' and is not selected. Below this radio button are three options: 'Previous version', 'Number of days', and 'Reference branch'. Each option has a description and a recommendation. At the bottom of the form are two buttons: 'Back' and 'Create project'.

Step 7 :- Now Select Analysis Method As Locally

Analysis Method

Use this page to configure and set up the way your analysis is performed.

How do you want to analyze your repository?

 Write Jenkins

 Write GitHub Actions

 Write Bitbucket Pipelines

 Write Circle CI

 Write Azure Pipelines

 GitHub CI

Scheduled integration with your workflow no matter which CI tool you're using.

 Locally

Use this for testing or advanced user cases. Other modes are recommended to help you set up your CI environment.

Step 8 :- Generate Token

Analysis Method: Locally

Analyze your project

We initialized your project on SonarCloud, now it's up to you to launch analysis!

1 Provide a token

Generate a project token

Copy existing token

Token name is

Expires in

total-sonar-project*

30 days

Generate

Please note that this token will only allow you to analyze this current project. If you want to add this same token to analyze multiple projects, you need to generate a global token in your [user account](#). See the [documentation](#) for more information.

This token is used to identify you when an analysis is performed. If it has been compromised, you can revoke it at any point in time in your [user account](#).

 Run analysis on your project

Step 9 :- After Generating Token Continue

Analysis Method: Locally

Analyze your project

We analyzed your project on SonarQube, now it's up to you to identify analysis!

1 Provide a token

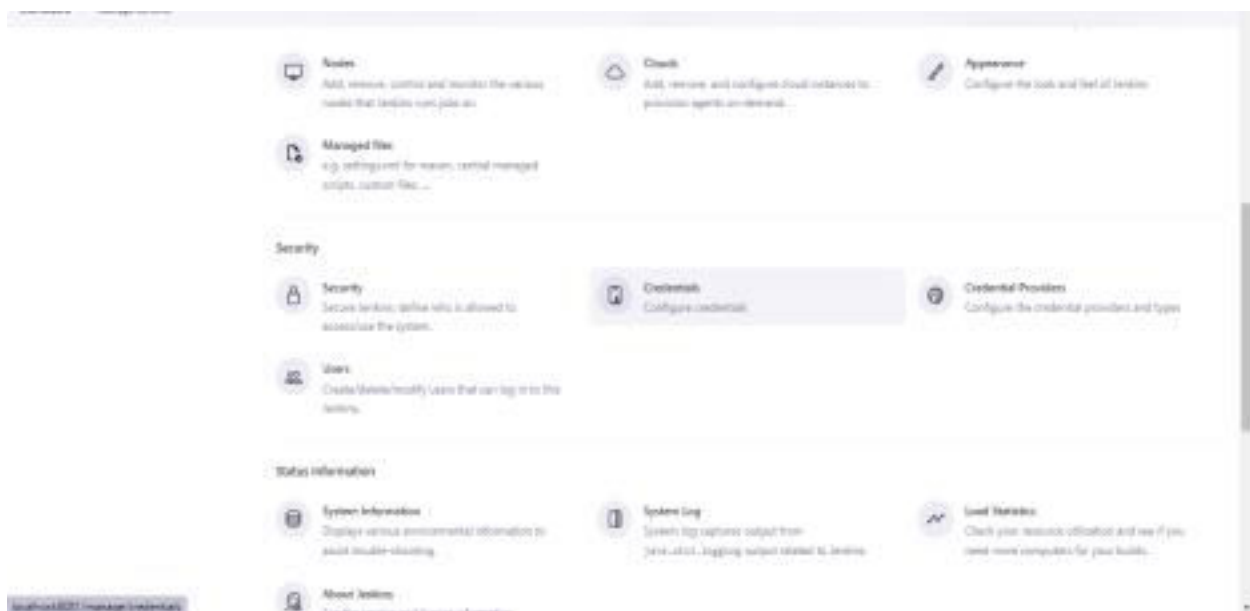
"token: token: project": xgp_009f187938ae2e543e6e8cc8e0e6a2079e44 

This token is used to identify you when an analysis is performed. If it has been compromised, you can revoke it at any point in time in your [user account](#).

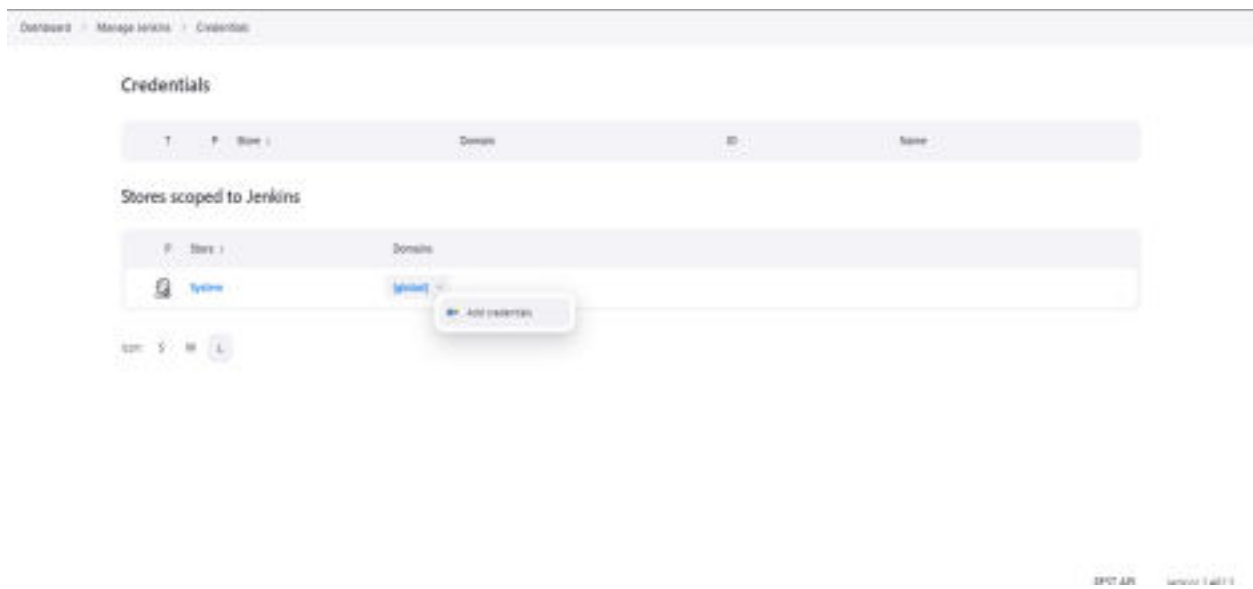
[Continue](#)

2 Run analysis on your project

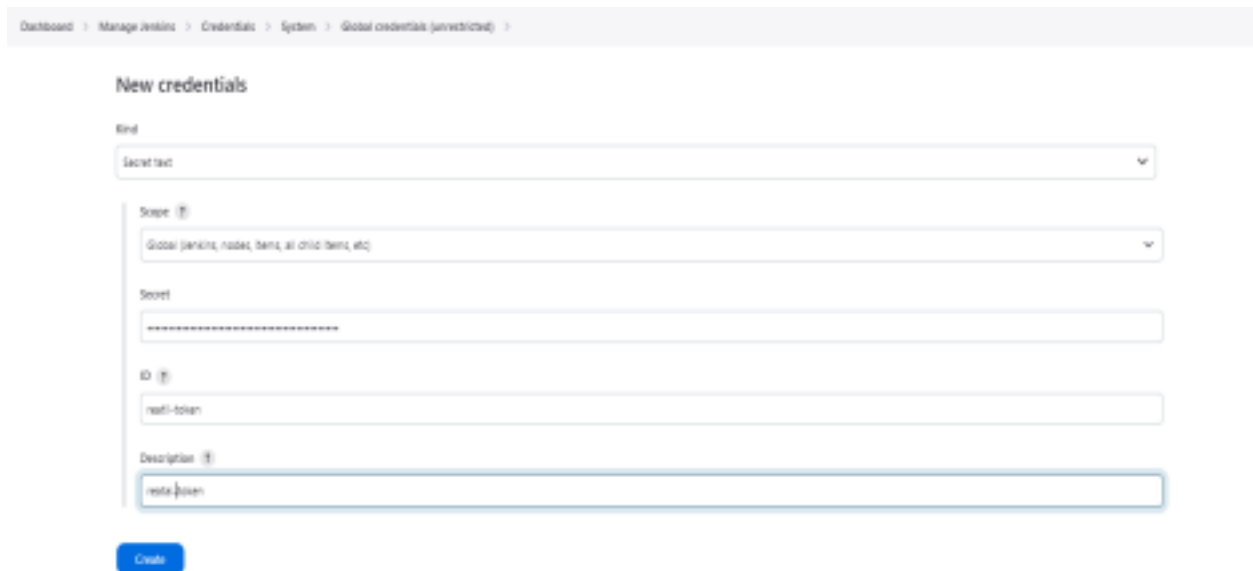
Step 10 :- Now In Jenkins Go To Manage Jenkins/Credential :-



Step 11 :- Now Go To Add Credential



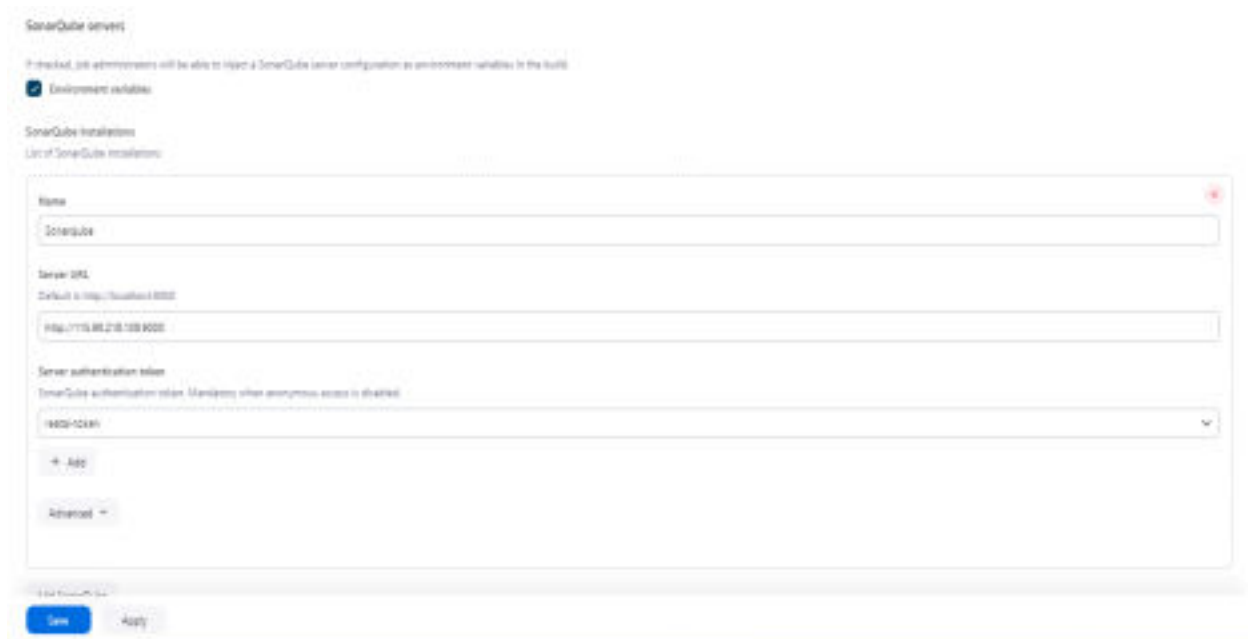
Step 12 :- Now Select “Kind” as Secret Text and Enter Your Token



Step 13 :- Now You can see that Your Credential Is Created



Step 14 :- Go To Manage Jenkins/System/Sonarqube Server



Step 15 :- Create A New Project

[Dashboard](#) > [All](#) > [New Item](#)

New Item

Enter an item name

Select an item type



Freestyle project
Classic, general-purpose job type that chains out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.



Maven project
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.



Pipeline
Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.



External Job
This type of job allows you to record the execution of a process run outside Jenkins, even on a remote machine. This is designed so that you can use Jenkins as a dashboard of your existing automation systems.

OK

Cancel



VIDYAVARDHINI'S COLLEGE OF ENGINEERING & TECHNOLOGY
DEPARTMENT OF INFORMATION TECHNOLOGY

K.T. Marg, Vasai Road (W), Dist-Palghar - 401202, Maharashtra

Step 16 :- Add Your Git Repository

Step 17 :- In Build Environment Select Prepare SonarQube Scanner Environment and enter your Token

Step 18 :- In Execute Sonar Scanner Add Your Analysis Properties

Step 19 :- Click On Build Now

Step 20 :- Console Output

Conclusion :

Integrating **Jenkins with SonarQube** allows organizations to automate the **SAST process**, ensuring that every code commit is checked for vulnerabilities and quality issues. This integration forms an essential part of **DevSecOps**, embedding security directly into the software development and delivery pipeline.